

Cpts 327 Solutions for Final

1: (10 Points) Let $p = 257$ and consider the multiplicative group $\mathbb{Z}_{257}^*$. We are looking for the smallest positive integer x such that $3^x \equiv 8 \pmod{257}$.

(a) (2 Points) **True or False:** The order of 3 in $\mathbb{Z}_{257}^*$ is 256.

Answer: Similar to HW5 Problem 5. 257 is prime, so $\varphi(257) = 256 = 2^8$. Fermat's little theorem induces $\text{ord}(3) \mid \varphi(257)$. Check and find $3^{128} \equiv -1 \pmod{257}$, so the order must be 256.

(b) (2 Points) Calculate $3^{64} \pmod{257}$.

Answer: Use the result of (c) to calculate.

(c) (2 Points) Calculate $3^{32} \pmod{257}$.

Answer: Use the result of (d) to calculate.

(d) (2 Points) Calculate $3^{16} \pmod{257}$.

Answer: Just calculate.

(e) (2 Points) Find x .

Answer: Similar to HW5 Problem 5. The result of (c) is 64. The result of (d) is -8 . Thus, $3^x \equiv (-1) \cdot 3^{16} \pmod{257}$. Use the result from (a), $3^{128} \equiv -1 \pmod{257}$, so $x = 128 + 16 = 144$. Due to order of 3 in this group is 256, 144 must be the smallest positive answer.

2: (10 Points) An embedded device implements RSA decryption using the Chinese Remainder Theorem (CRT). The notation here is the same as that in your HW6, where M is the plaintext, $M_p \in \mathbb{Z}_p, M_q \in \mathbb{Z}_q$ are the corresponding decrypted message during CRT and $d_p \in \mathbb{Z}_p, d_q \in \mathbb{Z}_q$ are the corresponding modular inverses for the secret key. An attacker uses a laser pulse to induce an error during the computation of $M_q = C^{d_q} \pmod{q}$. $M_p = 277 \pmod{q}$ and other processes are calculated correctly. As a result, the device outputs a corrupted plaintext $\widetilde{M}$ instead of the true plaintext M .

You have intercepted the following parameters from the device's public memory and its faulty output:

- Public Modulus $n = pq = 216793$ (assume that $p < q$)
- Public Exponent $e = 7$
- Captured Ciphertext $C = 195856$
- Corrupted "Plaintext" $\widetilde{M} = 776$

(a) (5 Points) Calculate p (assume that $p < q$).

Answer: Just calculate, similar to HW6 Problem 4.

(b) (5 Points) Calculate the original correct plaintext M .

Answer: Just calculate, similar to HW6 Problem 4.

3: (5 Points) Alice and Bob want to modify the textbook Diffie-Hellman protocol (the same as that in lecture and your HW5) with a large prime $p = 2^{129} + 17$ as the modulus. The modulus looks large and secure. And Alice wants to use a lucky number $h = 8$ as the base instead of the commonly used generator 2. In this case, the discrete log problem for 100 will be to solve x from $h^x \equiv 100 \pmod{p}$ instead of $2^x \equiv 100 \pmod{p}$.

(a) (2.5 Points) Alice chooses a private key $a = 43$, then obtains the public key is A . Calculate $A \pmod{17}$.

[Answer: Similar to HW5 Problem 7.](#)

(b) (2.5 Points) Bob's chooses a private key $b < a$ and calculate the public key B . Then Bob calculates the DH shared key K . A malicious attacker Eve eavesdrops $K = 289$. Help Eve to calculate $b \pmod{17}$.

[Answer: Similar to HW5 Problem 7. Note that \$17^2 = 289\$.](#)

4: (9 Points) 3 questions for the 3 labs:

- (3 Points) Look at the four functions written in C below:

```
void func_A(char *input) {
    char buf[64];
    int len = strlen(input);
    if (len <= sizeof(buf)) {
        memcpy(buf, input, len);
    }
}

void func_B(unsigned short size) {
    char buf[100];
    if (size < sizeof(buf)) {
        memset(buf, 'A', size);
        buf[size] = '\0';
    }
}

void func_C(char *input) {
    char buf[64];
    sscanf(input, "%s", buf);
}

void func_D(char *input) {
    char buf[64];
    buf[0] = '\0';
    strncat(buf, input, sizeof(buf) - 1);
}
```

Which function is vulnerable to a buffer overflow?

- (A) func_A
- (B) func_B
- (C) func_C
- (D) func_D

[Answer: C. Use sscanf without any size limitations.](#)

- (3 Points) Look at the four functions written in SQL below:

— *Option A:*

```
"SELECT * FROM products WHERE id = " + intval($input);
```

— *Option B:*

```
$order_by = ($input == "price") ? "price" : "name";
```

```
"SELECT * FROM products ORDER BY " + $order_by;
```

— *Option C:*

```
$safe_input = str_replace("'", "''", $input);
```

```
"SELECT * FROM users WHERE id = " + $safe_input;
```

— *Option D:*

```
$stmt = $db->prepare("SELECT * FROM users WHERE username = ?");
```

```
$stmt->execute([ $input ] );
```

Which implementation is vulnerable to SQL Injection?

- (A) Option A
- (B) Option B
- (C) Option C
- (D) Option D

Answer: C. 1 OR 1=1.

- (3 Points) In the context of Retrieval-Augmented Generation (RAG) mentioned in Lab3, the retrieval step often involves calculating the similarity or relevance between a user's query and a large corpus of documents. Which of the following is a widely used algorithm or scoring function for this purpose?

- (A) MCP
- (B) VICS
- (C) BM25
- (D) ICL

Answer: C. BM25 is introduced in Lab 3 lecture.

5: (10 Points) 4 questions for PKC:

- (2.5 Points) **True or False:** Diffie-Hellman (DH) is a Public Key Encryption algorithm, while Textbook RSA is not a true PKE because its encryption process is deterministic and lacks randomness.

Answer: F. Week 13 Notes. DH is not PKE. RSA is PKE. Reason is wrong.

- (2.5 Points) **True or False:** By appending a uniformly chosen fresh random bit string which is sufficiently long against brute-force attack to introduce randomness for each encryption, this variant of Textbook RSA can achieve CPA security (IND-CPA).

Answer: T. HW6 Problem 8.

- (2.5 Points) **True or False:** Textbook ElGamal encryption is not a CPA-secure Public Key Encryption due to its encryption process is deterministic and lacks randomness; however, by appending a uniformly chosen fresh random bit string which is sufficiently long against brute-force attack to introduce randomness for each encryption, this method can transform it into a CPA-secure scheme.

Answer: F. HW5 Problem 8. ElGamal is CPA and has randomness.

- (2.5 Points) **True or False:** The security of the Diffie-Hellman Key Exchange protocol relies on the Discrete Logarithm Problem (DLP); therefore, any group where the DLP is hard can ensure a secure DH Key Exchange protocol.

Answer: F. Week 13 Notes, Apr 6. DDH instead of DLP. DLP is necessary condition.

6: (8 Points) 2 questions for PRG/PRF:

- (4 Points) Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ be a secure PRG with fixed polynomial $\ell(n) > n$. Alice constructed a $G' : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ where for any seed $s \in \{0, 1\}^n$, the output is defined as:

$$G'(s) := \begin{cases} G(s) & \text{if the Hamming weight of } G(s) \text{ is even} \\ G(s) \oplus (0^{\ell(n)-1} \| 1) & \text{if the Hamming weight of } G(s) \text{ is odd} \end{cases}$$

Is G' a secure PRG?

- **A. Yes.** The proof is by reduction, i.e., assume there exists a PPT adversary $\mathcal{A}$ distinguishes G' from random, we can build a distinguisher $\mathcal{B}$ for G that flips the last bit of its challenge string if its weight is odd and then invokes $\mathcal{A}$.
- **B. Yes.** The proof is by reduction, i.e., assume there exists a PPT adversary $\mathcal{D}$ can break a single instance of G , we can trivially extend this attack by demonstrating that the distribution of $G'(s)$ remains computationally indistinguishable from $U_{\ell(n)}$.
- **C. No.** The construction violates the definition.
- **D. No.** The construction allows for a distinguisher with non-negligible advantage in polynomial time by observing the last bit.
- **E. No.** The construction allows for a distinguisher with non-negligible advantage in polynomial time by observing the hamming weight.

Answer: E. Construct a distinguisher to check the hamming weight, if from the PRG, always even; if from random, even or odd.

- (4 Points) Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ ($\ell(n) > n > 1$) be a secure PRG. Alice constructed a $G' : \{0, 1\}^{n+\ell(n)} \rightarrow \{0, 1\}^{\ell(n)+1}$ where for an input (s, r) with $s \in \{0, 1\}^n$ and $r \in \{0, 1\}^{\ell(n)}$, the output is defined as:

$$G'(s, r) := r \parallel \langle G(s), r \rangle$$

where $\langle \cdot, \cdot \rangle$ denotes the inner product modulo 2.

Is G' a secure PRG?

- **A. Yes.** The proof is by reduction, i.e., assume there exists a PPT adversary $\mathcal{A}$ distinguishes G' from random, we can build a distinguisher $\mathcal{B}$ for G that uses $\mathcal{A}$'s ability to predict the inner product bit to break the underlying security of G .
- **B. Yes.** The proof is by reduction, i.e., assume there exists a PPT adversary $\mathcal{D}$ can break a single instance of G , we can trivially extend this attack by re-sampling r to perform a frequency analysis on the inner product results.
- **C. No.** The construction violates the definition.
- **D. No.** The construction allows for a distinguisher with non-negligible advantage in polynomial time by observing the exposed r part and solve the inner product to obtain $G(s)$.
- **E. No.** The construction allows for a distinguisher with non-negligible advantage in polynomial time by observing the inner product part and solve a knapsack problem (or subset sum problem).

Answer: C. $n > 1 \Rightarrow n + \ell(n) > 1 + \ell(n)$. Should be alert to inner product.

7: (20 Points) 8 questions for PRG/PRF:

- (2.5 Points) **True or False:** A Pseudorandom Function (PRF) $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is by definition a deterministic algorithm.
Answer: T. Week 6 Notes, Feb 20.
- (2.5 Points) **True or False:** $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is a secure PRF, if and only if for any given input k and x , the output of $F_k(x)$ must be statistically indistinguishable from a uniformly chosen element from $\mathcal{Y}$.
Answer: F. Week 6 Notes, Feb 20.
- (2.5 Points) **True or False:** Let F be a secure PRF. We construct $G_k(x) = F_k(x) \oplus x$. $G_k(x)$ is a secure PRF.
Answer: T. Similar to Midterm Problem 9, but different question.
- (2.5 Points) **True or False:** For an encryption scheme $E = (Gen, Enc, Dec)$ with message space $\mathcal{M}$, key space $\mathcal{K}$ and ciphertext space $\mathcal{C}$, according to Shannon's Theorem, it is perfectly secret if and only if satisfying: (1) $|\mathcal{M}| = |\mathcal{K}| = |\mathcal{C}|$; (2) the key k is chosen uniformly at random from the key space $\mathcal{K}$.
Answer: F. Week 1-3 Notes.
- (2.5 Points) **True or False:** Any PRF family $F_s : \{0, 1\}^k \rightarrow \{0, 1\}$ for $s \in \{0, 1\}^n$ gives rise to a PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^m$ constructed by $G(s) = (F_s(1), F_s(2), \dots, F_s(m))$ as long as $n < 2^k$ (assuming the distribution on keys s is uniform).
Answer: T. Inverse logic of HW3 Problem 7. GGM taught us how to construct PRF using PRG like a binary tree. So if the tree already exists, we can sequentially go through its leaves to obtain a PRG.
- (2.5 Points) **True or False:** Let $F'_s : \{0, 1\}^k \rightarrow \{0, 1\}$ be a secure PRF family with a 1-bit output. We can obtain a new PRF family $F_s : \{0, 1\}^{k+\lceil \log \ell \rceil} \rightarrow \{0, 1\}^\ell$ for input $x \in \{0, 1\}^{k+\lceil \log \ell \rceil}$ with an ℓ -bit output by defining the evaluation as:

$$F_s(x) := (F'_s(x, 1), F'_s(x, 2), \dots, F'_s(x, \ell))$$

where the integers $1, \dots, \ell$ are identified with their $\lceil \log \ell \rceil$ -bit binary expansion (for small numbers like $(2)_{10} = (10)_2$, padding $(10)_2$ with sufficient 0 to maintain the length to be $\lceil \log \ell \rceil$).

Answer: F. $(x, \ell) \in \{0, 1\}^{k+2\lceil \log \ell \rceil}$ does not match the PRF input length requirement.

- (2.5 Points) **True or False:** Let $\mathbb{G}$ be a group of prime order q with a generator g . Given a $g^a \in \mathbb{G}$ and a secret seed $b \in \mathbb{Z}_q$, we define a length-doubling function as $G_{g^a}(b) = (g^b, g^{ab})$. This $G_{g^a}(\cdot)$ is a secure PRG if and only if the Discrete Logarithm Problem (DLP) is hard in $\mathbb{G}$.
Answer: F. Week 13 Notes, Apr 6. DDH instead of DLP.
- (2.5 Points) **True or False:** Let $G_{g^a}(b) = (G_0(g^a, b), G_1(g^a, b))$ be a secure PRG. We can construct a function family indexed by $a_0, \dots, a_k \in \mathbb{Z}_q$ as:

$$F_{a_0, \dots, a_k}(x) := G_{g^{a_k}}^{x_k} (G_{g^{a_{k-1}}}^{x_{k-1}} (\dots G_{g^{a_1}}^{x_1} (g^{a_0}) \dots))$$

for any $x \in \{0, 1\}^k$, where x_i selects the corresponding output half of G . This function family is a secure PRF under the same hardness assumption as the underlying PRG.

Answer: T. HW3 Problem 7, GGM construction.

8: (14 Points) 4 questions for CPA:

- (3.5 Points) **True or False:** If an encryption scheme satisfies IND-CPA security, it will also satisfy perfect secrecy. However, if a scheme satisfy perfect secrecy, it may not satisfy IND-CPA security due to IND-CPA is designed for the encryption of multiple messages in real world instead of only one.

Answer: F. That's why we transform from Week 3 to Week 4.

- (3.5 Points) **True or False:** Let F be a secure PRF. We define an encryption scheme as $Enc_k(m) = F_k(r) \oplus m$, where r is a public counter that increments with each encryption. This scheme is CPA-secure.

Answer: T. Week 7 Notes, Feb 23.

- (3.5 Points) **True or False:** In the public-key setting, any deterministic encryption scheme (e.g., Textbook RSA) can never be CPA-secure. However, in the private-key setting, a deterministic scheme can be CPA-secure if the key is always kept secret.

Answer: F. Totally fabricated statement.

- (3.5 Points) **True or False:** Consider a Pseudorandom One-Time Pad (P-OTP) scheme where $Enc_k(m) = m \oplus G(k)$ ($k \in \{0, 1\}^n, m \in \{0, 1\}^t$) for a secure PRG $G : \{0, 1\}^n \rightarrow \{0, 1\}^t$. Since $G(k)$ is computationally indistinguishable from a truly random string, this scheme satisfies IND-CPA security.

Answer: F. Week 7 Notes, Feb 23. Still deterministic.

9: (8 Points) 2 questions for PKC/CPA:

- (4 Points) Consider an encryption scheme $\Pi = (Gen, Enc, Dec)$ over a cyclic group $\mathbb{G}$ of prime order q with generator g . Assume the DDH assumption holds on $\mathbb{G}$. The key generation is: $pk = h = g^s$ and $sk = s$. For a message $m \in \mathbb{G}$, the encryption algorithm is: the encryption algorithm derives r using a secure PRF F such that $r = F_k(m)$, where k is a secret key shared between Alice and Bob. The ciphertext is:

$$Enc_{pk,k}(m) = (g^{F_k(m)}, h^{F_k(m)} \cdot m)$$

Is this scheme IND-CPA secure?

- **A. Yes.** The scheme is CPA-secure because F is a secure PRF, making the derived value r computationally indistinguishable from a uniform random value to any PPT adversary.
- **B. Yes.** The scheme remains CPA-secure under the Decisional Diffie-Hellman (DDH) assumption.
- **C. No.** The scheme is not CPA-secure and we can construct a polynomial-time adversary by observing the output. Because although the DDH assumption holds, the proof of IND-CPA security requires the hardness of discrete log problem (DLP) holds on the cyclic group $\mathbb{G}$.
- **D. No.** The scheme is not CPA-secure and we can construct a polynomial-time adversary by observing the output. Because the encryption algorithm incorrectly uses the PRF F to produce the exponents.

Answer: D. Incorrectly use the deterministic PRF to derive r . r should be randomness to ensure CPA security.

- (4 Points) Consider a modified textbook RSA-based encryption scheme. The public key is (n, e) and the private key is d . To encrypt a message $m \in \mathbb{Z}_n^*$, the encryption algorithm first picks a fresh random $r \in \mathbb{Z}_n^*$ and computes the ciphertext:

$$C = (r^e \pmod{n}, (r \cdot m)^e \pmod{n})$$

Assume all the required hardness assumption holds. Is this scheme IND-CPA secure?

- **A. Yes.** The scheme is CPA-secure because the use of a fresh random r for each encryption ensures that the ciphertext is non-deterministic, effectively masking the message m within the second component.
- **B. Yes.** The scheme is CPA-secure because an adversary would need to solve two RSA problem (computing e -th roots) for the two components to recover any information about m . Two RSA is better than one.
- **C. No.** The scheme is not CPA-secure because the adversary can obtain the secret key d to decrypt the first component r^e to find r , and then use r to recover m from the second component.
- **D. No.** The scheme is not CPA-secure because we can always retrieve one textbook RSA ciphertext from the two RSA ciphertexts provided.

Answer: D. Since one textbook RSA can be retrieved, textbook RSA is deterministic which is not CPA-secure.

10: (11 Points) RSA < –?– > FACTOR

- (2.5 Points) **True or False:** We can prove that if someone can successfully solve the RSA problem with modulus $n = pq$ (p, q are prime numbers) in polynomial time, then he/she can use the solver algorithm as a black-box to factor the modulus n in polynomial time.

Answer: F. Still an open problem. If you are interested in it, use Google search to explore with the keyword "Breaking RSA May Be Easier Than Factoring".

- (2.5 Points) **True or False:** We can prove that if someone can successfully factor a number $n = pq$ (p, q are prime numbers) in polynomial time, then he/she can use the factoring algorithm as a black-box to solve any RSA encryption scheme using the modulus n in polynomial time.

Answer: T. You can try to do a reduction. Easy to prove.

- (6 Points) A student attempts to provide a reduction to prove that if someone can successfully solve the RSA, then he/she can factor the modulus n in polynomial time. Assume the student has a RSA solver oracle denoted as $m \leftarrow \mathcal{O}(n, e, c)$ such that $m^e \pmod{n} \equiv c$. And the student can also access a test oracle denoted as $True/False \leftarrow \mathcal{O}'(n, e)$ to test whether $\gcd(e, \phi(n)) = 1$.

Inspired by quadratic residue and CRT under the modulus n , the student finds that for $x^2 \equiv a \pmod{n}$ where $n = pq$, using CRT to solve it can obtain four possible answers (i.e., respectively solve under $\pmod{p}$ and $\pmod{q}$). So if the student obtains two answers x and x' , it is possible to calculate $\gcd(x - x', n)$ to obtain the factoring of n .

Thus, the student wants to apply this technique from quadratic residue in the reduction. The student proposes the following reduction algorithm $\mathcal{R}$ to factor a given modulus $n = pq$ (p, q are prime numbers):

Loop:

- Pick a valid RSA public exponent e .
- Test whether $\gcd(e, \phi(n)) = 1$ using $\mathcal{O}'(n, e)$. If not, fail and goto Step (i).
- Choose a random element $x \in \mathbb{Z}_n^*$.
- Compute the ciphertext $c = x^e \pmod{n}$.
- Call the RSA oracle $\mathcal{O}(n, e, c)$ to obtain a value x' . If fail, goto Step (i).
- Test whether $(x')^e \equiv c \pmod{n}$. If not, fail and goto Step (i).
- Test whether $x' \neq x$. If not, fail and goto Step (i).
- Compute $\gcd(x - x', n)$ to reveal a factor of n , denoted as m . Test whether $m|n$. If not, fail and goto Step (i).

ix. Test whether $m \neq 1$ and $m \neq n$. If not, fail and goto Step (i).

Which of the following statement is True?

- **A.** The proof has some flaws: Step (v) is invalid because in the security game definition, RSA oracle can only be queried on ciphertexts that the adversary does not already know the plaintext for. But in Step (iii) it already knows.
- **B.** The proof is correct.
- **C.** The proof has some flaws: Step (ii) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.
- **D.** The proof has some flaws: Step (v) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.
- **E.** The proof has some flaws: Step (vi) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.
- **F.** The proof has some flaws: Step (vii) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.
- **G.** The proof has some flaws: Step (viii) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.
- **H.** The proof has some flaws: Step (ix) will be executed but always fail, thus the loop will never end. And the steps after it will never be executed.

Answer: F. Finding square roots modulo n is a many-to-one mapping (specifically 4-to-1 for RSA moduli), whereas the RSA power function $x^e \pmod{n}$ is a bijection. That's why we need $\gcd(e, \phi(n)) = 1$.